Project Report - ECE 176

Alexey Grishakov
Electrical and Computer engineering
A17999821

Abstract

I test whether a Grassmann-flow language model can work as an attention free alternative to a standard causal Transformer at around 100M parameters. Instead of token to token self-attention, the Grassmann model uses causal geometric mixing with Plücker coordinates computed from low dimensional token subspaces. Both models use the GPT-2 tokenizer and the same training setup so the comparison is mostly about the mixing mechanism itself.

At step $9{,}500$ (about 10 Billion tokens, same dataloaders for both), the Grassmann model gets loss 5.8413 and perplexity 344.23, versus Transformer loss 6.1234 and perplexity 456.41. Training total was $12{:}42$ for Grassmann and $24{:}06$ for the Transformer, so roughly $1.9\times$ faster. That said, when I profiled inference with KV caching the compute cost difference between the two was not that large, so the win is mainly in quality and training speed rather than inference cost.

1 Introduction

Most autoregressive language models today use self-attention, which scales quadratically with sequence length. As context windows get longer and models get bigger, this becomes expensive in both memory and compute. The question I wanted to answer in this project is whether it is possible to abandon attention entirely using a geometric flow approach and still get reasonable language modeling quality.

The architecture I test comes from "Attention Is Not What You Need" [1], which proposes replacing attention with Grassmann-flow geometric mixing over token subspaces. Tokens are projected to a lower dimensional subspace, causal interactions between positions are captured via Plücker coordinates, and a learned geometric mixer takes the place of attention in each block. I wanted to go beyond just reproducing the mechanism and actually test it at a nontrivial scale (~ 100M parameters).

This report compares that Grassmann model against a causal Transformer baseline trained with the same tokenizer and data pipeline. I report unseen data loss/perplexity, total training time, inference runtime, and profiled compute scaling.

2 Related Work

The main reference for the Grassmann model is the original paper by Chong et al. [1], which introduces the idea of replacing self-attention with causal geometric mixing over subspace representations. For the baseline, I use a GPT-style causal Transformer since it is still the standard architecture for language modeling at this parameter scale. Training data comes from a FineWeb derived corpus [2], which is one of the commonly used large scale web text datasets.

Some earlier work on language modeling benchmarks has also shown that prediction quality can depend on long range discourse context and not just local n-grams [3]. I kept this in mind when designing the evaluation: both models share the same tokenizer and data pipeline, and were trained

on the exact same set of tokens, in the exact same order, so the comparison is as direct as I could make it.

3 Method

3.1 Model setup

Both models use the GPT-2 tokenizer ($50{,}257$ vocabulary), context length 1024, and tied input/output token embeddings. The main Grassmann run has approximately 102.47M parameters with:

- $d_{\text{model}} = 640$, $n_{\text{layers}} = 16$, $d_{\text{ff}} = 2560$,
- geometric rank $r = 24$, projection width $d_{\text{geom}} = 256$,
- causal offsets $\{1, 2, 4, 8, 16, 32, 64\}$.

The Transformer baseline is size matched and trained with the same data pipeline and optimizer schedule for a fairer comparison.

3.2 Causal Grassmann block

For each token position t, the model computes geometric features only from past offsets $t - \Delta$, preserving autoregressive causality. Given reduced vectors $z_{t-\Delta}$ and z_t, pairwise antisymmetric products define a Plücker feature vector:

$$p_{ij}^{(\Delta)}(t) = z_{t-\Delta,i} z_{t,j} - z_{t-\Delta,j} z_{t,i}, \quad i < j.$$

These normalized geometric features are projected back to model width, aggregated across offsets, and mixed with the current hidden state through a learned gate, followed by a standard FFN residual path.

3.3 Training and evaluation protocol

Both models are trained with the same tokenization, sequence length, and held out (unseen data) evaluation windows so the comparison is controlled. I measure held out loss and perplexity on the same test windows for both. I also run context sensitivity sweeps (varying how much prefix the model can see), total runtime benchmarks, and profiled FLOP counts at different prompt lengths. For the compute profiling, both models use KV-cached decode so the numbers reflect actual autoregressive inference.

4 Experiments

4.1 Datasets and data format

- **Training data:** FineWeb derived token stream (GPT-2 tokenization, pre tokenized)[2].
- **Held out evaluation:** Fixed token windows from `data/test_tokens.bin` with starts from `data/test_starts.npy`.
- **Evaluation approach:** shared sequence length 1024, device CUDA, bfloat16 evaluation.

4.2 Benchmarks

- `eval.py` measures held-out loss/perplexity, a short context sweep, and prompt-generation speed on the same checkpoints.
- `bench.py` measures synthetic-prompt prefill latency and uncached decode throughput across prompt lengths.
- `compute_profile.py` profiles prefill and KV-cached decode FLOPs as prompt length increases.

4.3 Head to head summary

Table 1 summarizes the main checkpoint metrics from `polished_metrics.csv`. The Grassmann model is better on held-out loss and perplexity, evaluates faster on the shared held-out set, and reaches its best context-bucket loss at the full 1024-token window.

Table 1: Main comparison metrics at step 9,500, taken directly from `polished_metrics.csv`. Grassmann lowers held-out perplexity by about 24.6%, while the Transformer remains much faster in greedy generation throughput.

Model	Params (M)	Loss	Perplexity	Eval sec	Best ctx	Greedy tok/s
Grassmann	102.47	5.8413	344.23	528.09	1024	73.57
Transformer	111.60	6.1234	456.41	835.58	256	197.31

4.4 Training time and runtime comparison

Training was noticeably faster for the Grassmann model: 12:42 (hours:minutes) versus 24:06 for the Transformer. That is about $1.90\times$ faster, or a 47.3% reduction in total training time.

Inference is a different story. At short prompts the Transformer is clearly faster, but the gap shrinks as context length grows and actually reverses at 1024 tokens in my benchmark.

Figure 1 comes from `bench.py`, which uses synthetic token prompts, batch size 1, and 64 generated tokens per decode run. The plot shows a split result: the Transformer is much faster for short prompts and still leads on uncached decode throughput, but Grassmann's prefill curve grows more gently and is faster by the longest tested prompt (24.96 ms vs. 36.22 ms at length 1024).

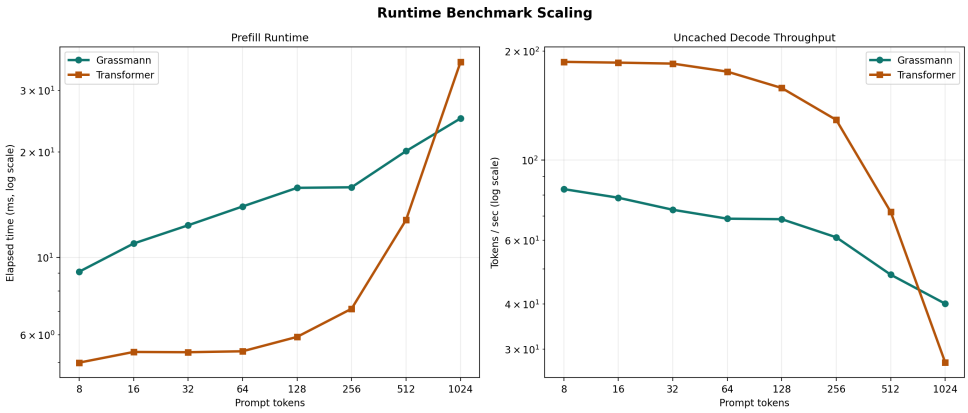

Figure 1: Runtime scaling from `bench.py`. Left: prefill latency versus prompt length. Right: uncached decode throughput for 64 generated tokens.

So there is a trade-off here. The Transformer is faster at short contexts, probably because its attention kernels are more optimized at this point. But Grassmann catches up as the prompt gets longer, and at 1024 tokens it is actually faster on prefill. The practical speed advantage only really shows up near the top of the tested range, though. This possibly means that further testing on models trained with a significantly larger context window would be necessary to see if the Grassmann model can truly converge on the linear scaling, as opposed to the quadratic scaling of the Transformer.

4.5 Context sensitivity comparison

Both models were evaluated by scoring a fixed suffix while varying the visible prefix length. Grassmann is more stable across context lengths, while the Transformer shows a larger gain from increasing context before flattening out.

Grassmann has better loss at every tested context length, from 5.8317 at 32 tokens to 5.8302 at 1024, which is good. But it is also interesting that its numbers barely change as context grows: the spread

from 32 to 1024 is tiny. The Transformer, on the other hand, improves more noticeably from 6.1180 at 32 to 6.0990 at 256 before it levels off and slightly worsens again by 1024. So the Grassmann model might not be using long-range context as effectively as the Transformer, even though its absolute quality is better.

This matters because prior work [3] has shown that some predictions really do depend on long-range discourse context rather than just local patterns. That concern is also consistent with the current implementation: the model uses pre-defined look-behind offsets, so it cannot directly mix information from arbitrarily far back in the sequence. This creates a real limitation for larger context windows, because tokens outside that offset pattern are effectively invisible to the geometric mixer.

4.6 Profiled compute scaling with KV cache

To separate raw runtime from operation-level compute cost, I also profiled FLOPs as prompt length increases. Figure 2 comes from `compute_profile.py`; its prefill panel runs out to 1024 tokens, while the cached decode panel profiles 16 generated steps out to prompt length 256. The decode study uses the cached autoregressive path for both models, so it reflects inference with KV cache rather than naive recomputation. This is important, because without a KV cache my implementation of the Grassmann flow model would drift back toward quadratic behavior instead of the intended linear-in-context decode path. The two models track each other closely in both panels: at length 1024, prefill is 254.64 GFLOPs for Grassmann versus 270.07 GFLOPs for the Transformer, while at decode length 256 Grassmann is slightly higher (66.40 vs. 62.92 GFLOPs).

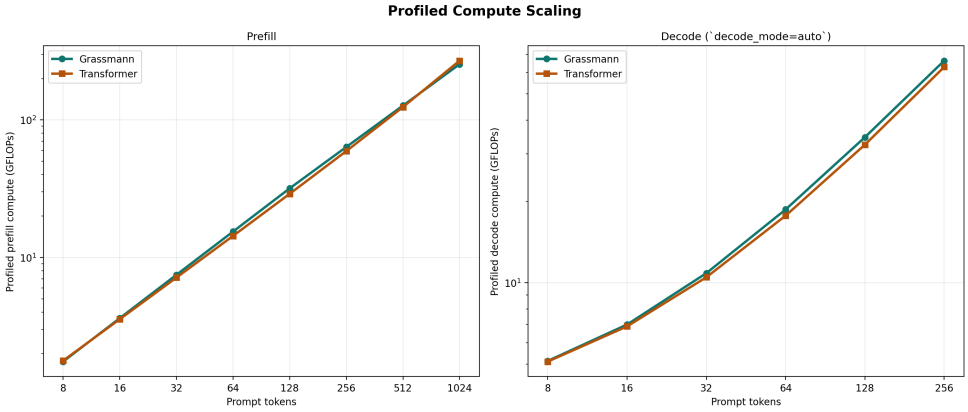

Figure 2: Profiled compute scaling from `compute_profile.py`. Left: prefill FLOPs versus prompt length. Right: KV-cached decode FLOPs for 16 generated steps.

These profiles are worth looking at carefully because they tell a somewhat different story than the overall runtime numbers. When both models use KV cache, the FLOP difference between them is actually pretty small. Prefill FLOPs are close, and cached decode FLOPs are also close, with Grassmann even a bit higher at some prompt lengths. So the main gain from Grassmann in my experiments is not really about inference FLOPs; it is about quality and training speed.

4.7 Discussion

The final comparison shows that Grassmann flow is a viable attention free alternative at this scale. Its main benefits in this project are:

- better quality on held-out data than the size matched Transformer baseline; held out perplexity is improved by about 24.6%
- substantially shorter total training time in the current implementation,
- competitive long context runtime, with an advantage appearing at the longest tested prompt.

That said, there are clear caveats. Once both models use KV cache, the FLOP difference during decode is small, so the Grassmann model does not have a big inference compute advantage in practice.

It is also slower than the Transformer at short contexts, and the context sensitivity results suggest that it may not be taking full advantage of long range information in this checkpoint.

So I would say geometric mixing can beat a standard Transformer on quality and training efficiency at 100M parameters, but it is not an obvious win on inference cost. How much of that is fundamental versus just my implementation is hard to say. Testing with a larger context window would be a natural next step, since 1024 is still relatively short and the linear vs quadratic scaling difference should become more pronounced with longer sequences.

5 Supplementary Material

Additional materials for report:

- Training, evaluation, and benchmarking code written entirely by me (the original paper did not provide implementation code),
- Benchmark logs,
- Demo video

References

[1] Z. Chong, "Attention is not what you need," 2025. [Online]. Available: https://arxiv.org/abs/2512.19428

[2] G. Penedo, H. Kydlíček, L. B. allal, A. Lozhkov, M. Mitchell, C. Raffel, L. V. Werra, and T. Wolf, "The fineweb datasets: Decanting the web for the finest text data at scale," in *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. [Online]. Available: https://openreview.net/forum?id=n6SCkn2QaG

[3] D. Paperno, G. Kruszewski, A. Lazaridou, Q. N. Pham, R. Bernardi, S. Pezzelle, M. Baroni, G. Boleda, and R. Fernández, "The lambada dataset: Word prediction requiring a broad discourse context," 2016. [Online]. Available: https://arxiv.org/abs/1606.06031